MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (AUTONOMOUS), HYDERABAD
Department of Emerging Technologies

Paddy Leaf Disease Detection for a Low-Cost Phone

Complex Engineering Problem Task
CM701PC Deep Learning and CM751PC Deep Learning Lab
B.Tech CSE (AI&ML), VII Semester, Regulation MR-22

Task number: 1 (Crop leaf disease detection on a low-cost device)
Team number: 5 | **Variant:** 1B, Paddy leaf diseases
Submission name: DL_T05_1B

Team members and contribution

Roll number	Role allotted by the department	Contribution
23261A6611	Data lead	Led overall project development, dataset preparation, data splitting and leakage checks, model development, experiments E1–E7, training, evaluation, E7 selection and TFLite conversion.
23261A6658	Model lead	Supported model architecture analysis, reviewed MobileNetV2 configurations, training parameters, augmentation techniques and TFLite deployment requirements.
23261A6606	Evaluation and report lead	Supported performance analysis, confusion matrix interpretation, result verification, figure preparation, report documentation and reference checking.

1. Problem understanding

A farmer organisation needs an application that identifies paddy leaf diseases from a single leaf image. The application is intended to work offline on an entry-level Android phone. Therefore, the model should be small, provide fast predictions and maintain good classification performance even with limited and imbalanced training data.

The project focuses on four performance requirements: the TensorFlow Lite (TFLite) model size should be below 10 MB, prediction time should be below 300 ms on one CPU core, test macro F1-score should be at least 0.80, and recall should be at least 0.75 for each of the two rare classes: bacterial panicle blight and bacterial leaf streak.

The main design challenges are balancing model accuracy, size and speed; selecting suitable data augmentation; handling class imbalance; and reducing the effect of image-quality changes on classification. The trade-offs identified at the start were: a larger network raises accuracy but threatens the size and speed limits; heavier augmentation can improve robustness but may crop small lesions; class weights should protect the rare classes but could lower overall F1; and quantisation reduces file size but may change accuracy or speed. Each of these was tested by an experiment in Section 3.

2. Data preparation and exploratory analysis

We used the Paddy Doctor dataset from Kaggle [3], which contains 10,407 labelled paddy leaf images belonging to 10 classes (nine disease classes and one normal class). Each image has an original resolution of 480 × 640 pixels. The two smallest classes are bacterial_panicle_blight (337 images) and bacterial_leaf_streak (380 images), which were treated as rare classes. The normal class is the largest (1,764 images, about 5.2 times the smallest class). The images contain natural backgrounds and lighting variations, as Figure 1 shows.



Figure 1. One sample image per class from the Paddy Doctor dataset (rare classes marked RARE).

The dataset was divided into training (70%), validation (15%) and testing (15%) sets using stratified sampling with random seed 42. An MD5 hash check was performed to identify exact duplicate images and reduce data leakage. Four duplicate images within the test set were retained, while 12 training/validation images identical to test images and 22 training images identical to validation images were removed. The final dataset contained 7,252 training images, 1,559 validation images and 1,562 test images. All images were resized to 224 × 224 pixels. Data augmentation was applied only to the training set, and the test set was reserved for final evaluation. Since near-duplicate images cannot always be identified using exact hashing, some similarity may still remain.

Simulated field-shift images: Real field photographs could not be collected within the available time and resources. Therefore, simulated field-shift datasets were created by applying image degradations to held-out images (Figure 2). Shift A includes JPEG compression, colour changes, haze, noise and blur. Shift B includes motion blur, salt-and-pepper noise, strong JPEG compression and low-light effects. These simulated images were used to study how image-quality changes affect model performance. They are not real field photographs and cannot fully represent actual farm conditions. Some Shift B degradations are similar to the augmentation techniques used during E7 training.

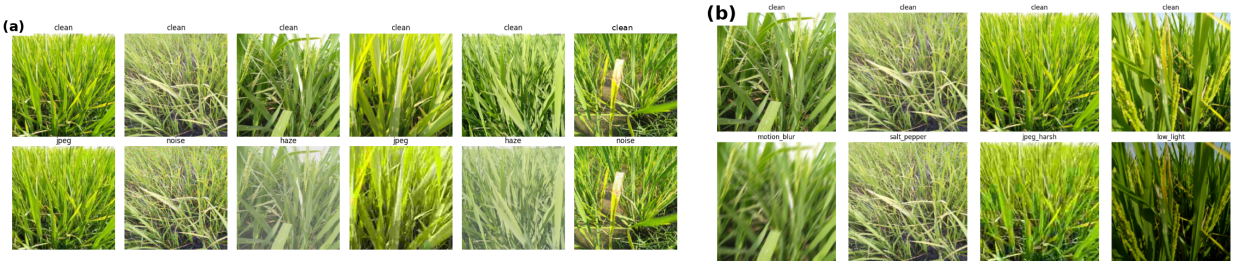


Figure 2. Simulated field-shift examples on validation images (top row clean, bottom row degraded): (a) Shift A (JPEG compression, noise, haze); (b) Shift B (motion blur, salt-and-pepper noise, harsh JPEG, low light).

3. Models and experiments

All design choices were judged on the validation split; the test split was used once, for the final model. Each experiment used one run unless stated, and class weights ("balanced") were on unless stated. All models use 224 × 224 inputs, the Adam optimiser, batch size 32 and early stopping on validation loss. The MobileNetV2 models were first trained with the backbone frozen (10 epochs, learning rate 1e-3) and then fine-tuned (learning rate 1e-4, up to 15 epochs, patience 4; E7: up to 30 epochs, patience 6). Pixel rescaling is inside the model, so the exported TFLite file accepts raw 0–255 pixel values. Seeds are fixed (42 for the split and training), and the code, split files, models and results are in the submitted ZIP (DL_T05_1B).

E3 augmentation: flips, rotation (about ±22°), zoom-in up to 25%, brightness and contrast ±20%. E7 augmentation: milder geometry (rotation about ±14°, zoom-in up to 10%, brightness and contrast ±15%) plus random Gaussian blur (sigma 0.5–2.0), Gaussian noise (standard deviation 5–20) and down-and-up resizing to 40–80% resolution, applied to about 55% of training images inside the model.

Table 1. Summary of experiments E1 to E7 on the validation split (one run each unless stated).

ID	Setup	Parameters	Validation macro F1	Recall: blight / streak
E1-old (two runs)	Scratch CNN, unseeded; provenance not recorded	99,690	0.366 and 0.668	0.60 / 0.702 and 0.88 / 0.930
E1 seeds 1 and 2	Scratch CNN with explicit seeds	99,690	0.570 and 0.735	0.44 / 0.737 and 0.82 / 0.772
E2	MobileNetV2-0.35, frozen backbone, augmentation	423,018	0.509	0.60 / 0.719
E3	E2 fine-tuned for 15 epochs with augmentation	423,018	0.826	0.90 / 0.965
E4	E3 without augmentation	423,018	0.890	0.88 / 0.930
E5	E4 without class weights	423,018	0.895	0.76 / 0.930
E6	E4 with width 0.5	719,034	0.886	0.84 / 0.860
E7	Milder geometric augmentation with blur, noise and low-resolution effects; up to 30 epochs, patience 6	423,018	0.909	0.92 / 0.912

The scratch CNN was run four times: twice without an explicit seed and twice with seeds 1 and 2. A third seeded run was not completed due to the available time. All four results are reported. Validation

macro F1 ranged from 0.366 to 0.735, showing considerable variation between runs (the seeds change only the initial weights). Therefore, small performance differences between experiments should be interpreted cautiously.

Findings. Fine-tuning the MobileNetV2 backbone improved validation macro F1 from 0.509 in E2 to 0.826 in E3. Removing augmentation in E4 increased clean validation macro F1 to 0.890. However, removing class weights in E5 reduced bacterial panicle blight recall from 0.88 to 0.76, even though overall macro F1 remained similar (0.895). The width-0.5 configuration in E6 did not improve performance despite 70% more parameters. These results show that fine-tuning was important for classification performance, while class weights helped protect recall for the rare disease class.

On the simulated shift validation sets, E3 and E4 showed a considerable reduction in macro F1 compared with clean validation images (for example, E4 fell from 0.890 to 0.527 on Shift A; Table 2). To improve robustness, E7 used milder geometric augmentation along with blur, noise and low-resolution effects.

Table 2. Validation macro F1 of E3, E4 and E7 on clean images and on the simulated Shift A and Shift B sets.

Model	Clean	Shift A	Shift B
E3	0.826	0.523	0.451
E4	0.890	0.527	0.429
E7	0.909	0.742	0.625

Decision rule: Before running E7, we decided to select it over E4 only if it achieved validation macro F1 of at least 0.80, recall of at least 0.75 for both rare classes, and a Shift B macro F1 at least 0.05 higher than E4. E7 satisfied all three conditions, achieving clean validation macro F1 of 0.909, rare-class recalls of 0.92 and 0.912, and Shift B macro F1 of 0.625 compared with E4's 0.429. Therefore, E7 was selected as the final model. However, E7 also differed from E4 in epoch limit and patience, so the individual contribution of each change was not measured separately.

4. Results against the performance targets (test split, final model E7)

Table 3. Performance targets and results of the final model (E7) on the test split.

Performance target	Required value	E7 result	Target met?
TFLite model size	Below 10 MB	1.66 MB (float32), 0.58 MB (dynamic int8)	Yes
Prediction time (one CPU thread)	Below 300 ms	2.6 ms (float32), 4.7 ms (dynamic int8)	Yes (Colab CPU proxy)
Test macro F1-score	At least 0.80	0.889 (TFLite float32: 0.8888; dynamic int8: 0.8832)	Yes
Recall: bacterial panicle blight	At least 0.75	0.882 (45 out of 51 images)	Yes
Recall: bacterial leaf streak	At least 0.75	0.912 (52 out of 57 images)	Yes

The selected E7 model met all five performance targets on the held-out test dataset. Both TFLite versions were below the 10 MB size limit. The measured inference times were also below 300 ms. However, latency was measured on a Colab CPU using one thread and excludes image decoding and resizing; it is not a measurement on an actual Android phone. The margin of about 100 times under the limit is what supports the claim.

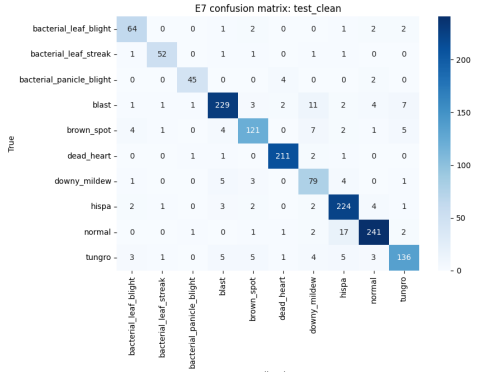


Figure 3. Confusion matrix of the selected E7 model on the held-out clean test dataset.

The float32 TFLite model achieved a test macro F1-score of 0.889. Recall for the two rare classes was 0.882 for bacterial panicle blight and 0.912 for bacterial leaf streak. Figure 3 shows the confusion matrix. Per-class precision, recall and F1-score and the confusion matrices for the clean test split and for both simulated field-shift versions of the test images are provided in the submitted results folder

(per_class_E7_*.csv and confusion_E7_*.png). The lowest class precision on clean test images was observed for downy mildew (0.731). The most frequent confusions were normal leaves classified as hispa (17 images) and blast classified as downy mildew (11 images).

Table 4. E7 on test images: clean and simulated field-shift sets (not real field photographs).

Data	Accuracy	Macro F1	Recall: blight	Recall: streak
Clean	0.898	0.889	0.882	0.912
Shift A	0.757	0.742	0.667	0.807
Shift B	0.638	0.611	0.569	0.737

The task statement sets no numeric target for field images. Panicle blight recall falls below 0.75 on both simulated sets, and leaf streak recall falls just below it on Shift B. On Shift B the model over-predicts leaf streak (precision 0.412), which keeps its recall up at the cost of false alarms.

5. Answers to the report questions

Q1. Why does convolution with parameter sharing suit this problem better than a fully connected network?

Convolutional Neural Networks (CNNs) are better suited for image classification because they learn local image features such as edges, textures and shapes. Convolution layers use small filters that move across the image, allowing the network to detect features at different locations. Parameter sharing reduces the number of trainable parameters compared with fully connected networks. In terms of the "infinitely strong prior" view, a convolutional layer is a fully connected layer whose weights are forced to be local and identical at every position, and pooling adds the prior that small shifts of a feature should not change the answer.

Pooling layers reduce feature-map dimensions and help the model become less sensitive to small changes in image position. These properties are useful for paddy leaf disease detection because disease symptoms, such as spots and lesions, may appear at different locations on a leaf.

For a 224 × 224 RGB image, a fully connected layer with 64 neurons would require approximately 9.6 million weights at the input layer alone. In comparison, our scratch CNN had 99,690 parameters, while the MobileNetV2-0.35 model had 423,018 parameters. This illustrates how convolution-based architectures can reduce model complexity. We did not train a fully connected baseline, so this comparison is based on architecture and parameter count rather than a direct performance experiment.

Q2. Is the lower accuracy on simulated field-shift images caused by overfitting, a domain shift, or both?

Both, with shift dominating. Table 5 gives accuracy on un-augmented training images, clean validation images and Shift B validation images.

Table 5. Training, validation and Shift B validation accuracy, and the gaps between them.

Model	Train	Val	Val Shift B	Train to val gap	Val to Shift B gap
E3	0.860	0.828	0.478	3 points	35 points
E4	0.994	0.895	0.441	10 points	45 points
E7	0.950	0.915	0.641	3.5 points	27 points

The results indicate that both overfitting and domain shift affect model performance, but the larger problem is the shift in image conditions. E4 shows a 10-percentage-point gap between training and clean validation accuracy, which suggests overfitting. However, its accuracy decreases by 45 percentage points on Shift B images. This larger reduction indicates that changes in image quality have a strong effect on classification.

E7 reduces the training-to-validation gap to 3.5 percentage points and the validation-to-Shift B gap to 27 percentage points. This suggests improved generalisation to the simulated image degradations. However, these results are based on code-generated images and do not establish performance under actual farm conditions.

Q3. Which single design choice made the largest difference?

Fine-tuning the MobileNetV2 backbone had a major impact on clean validation performance. Macro F1 increased from 0.509 in E2, where the backbone was frozen, to 0.826 in E3, where the backbone was fine-tuned (and from 0.681 to 0.890 in the no-augmentation recipe).

Class weighting was also important for rare-class detection. When class weights were removed in E5, bacterial panicle blight recall decreased from 0.88 in E4 to 0.76, even though overall macro F1

remained similar.

For robustness, E7 introduced milder geometric augmentation along with blur, noise and low-resolution effects. Its Shift B validation macro F1 increased from 0.429 in E4 to 0.625 in E7. However, E7 also used a different epoch limit and patience setting. Therefore, the improvement cannot be attributed to augmentation alone.

6. Limitations and further work

Most experiments were single runs, so small differences may be due to random variation; the scratch CNN varied from 0.366 to 0.735 validation macro F1. The rare-class recall values rest on only 51 (bacterial panicle blight) and 57 (bacterial leaf streak) test images. The simulated field-shift sets are code-generated, not real farm photographs, and may not represent all real-world conditions. E7 differs from E4 in augmentation, epoch limit and patience, whose individual effects were not measured. Inference time was measured on a Colab CPU, not on a low-cost Android phone, and excludes image decoding and resizing. Exact-duplicate removal may not identify all near-duplicate images.

Future work: collect real paddy leaf photographs under different lighting, backgrounds and cameras (including rare classes); repeat key experiments with multiple seeds; measure inference time on a low-cost Android phone; and run separate experiments for augmentation, epoch limit and patience.

7. References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.

[2] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[3] "Paddy Doctor: Paddy Disease Classification," Kaggle competition. [Online]. Available: https://www.kaggle.com/competitions/paddy-disease-classification (accessed Nov. 2026).

[4] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 265–283.

Appendix: declaration of AI tool use

AI tools were used during the project to support task understanding, experiment planning, Python and TensorFlow code development and debugging, interpretation of experimental results, and preparation of project documentation. The team reviewed and modified the generated suggestions, executed the code in Google Colab, checked the experimental outputs and prepared the final report based on the results obtained. All reported performance values are based on the team's recorded experiments. AI tools were used as assistance and did not replace the team's responsibility for implementation, verification and final submission.